

Instituto Tecnológico de Costa Rica

Ingeniería en Computadores

Algoritmos y Estructuras de Datos II

Conocimiento de Ingeniería

Grupo : 2

Profesor :

Leonardo Araya

Miembros de Grupo :

Alexs Eduardo Feng Wu

Jiacheng Tan He

Abril 8, 2025

Tabla de contenidos	
Tabla de contenidos	1
Introduccion	2
Descripción del Problema	2
Descripción de la Solución	2
Descripción Detallada de las Implementaciones	3
Diagrama UML	5
Link del repositorio de Github	5

Introducción

En el lenguaje C++, la administración de memoria dinámica es una responsabilidad del programador. Esto puede llevar a errores comunes como fugas de memoria, uso de memoria no inicializada, accesos a memoria liberada y referencias colgantes.

Este proyecto, MPointers 2.0, busca abstraer y automatizar este manejo de memoria mediante el uso de una clase genérica `MPointer<T>`, que actúa como un smart pointer personalizado, pero con una diferencia importante: la memoria no se administra localmente, sino en un servidor remoto, a través de gRPC.

La implementación también incluye funcionalidades como conteo de referencias, recolección de basura y desfragmentación de memoria, además de herramientas de monitoreo del estado de la memoria (dumps automáticos con timestamp).

Descripción del Problema

El desarrollo de aplicaciones robustas en C++ frecuentemente se ve afectado por errores derivados de una mala gestión de memoria dinámica. Los punteros tradicionales requieren una administración cuidadosa, lo que puede dificultar el mantenimiento del código y aumentar el riesgo de errores.

Asimismo, en entornos distribuidos, compartir y sincronizar memoria entre componentes puede volverse aún más complejo si no existe una estructura centralizada que controle su estado.

Por lo tanto, se plantea la necesidad de una solución que simplifique este proceso, proporcione control, y además, sea fácilmente auditable y extensible.

Descripción de la Solución

La solución desarrollada consiste en una clase llamada `MPointer<T>`, que reemplaza el uso de punteros tradicionales mediante una interfaz de alto nivel, segura y distribuida. Esta clase encapsula toda la lógica de gestión de memoria y comunicación con un servidor remoto (implementado también como parte del proyecto) que realiza las siguientes funciones:

- Reserva de bloques de memoria (Create).
- Escritura de valores (Set) y lectura (Get).
- Manejo de referencias compartidas (IncreaseRefCount, DecreaseRefCount).
- Eliminación automática de bloques sin referencias.
- Desfragmentación del espacio ocupado.
- Registro automático del estado de la memoria (dumps con fecha/hora exacta).

El sistema está diseñado para funcionar como una arquitectura cliente-servidor utilizando gRPC, lo que permite su expansión a múltiples clientes, posibles entornos en red, o incluso la virtualización de distintos espacios de memoria remota.

Esta solución no solo automatiza el manejo de memoria, sino que proporciona trazabilidad y control total sobre los bloques existentes, siendo útil tanto en aplicaciones educativas como en entornos reales donde se requiera gestión remota de recursos.

Descripción Detallada de las Implementaciones

Clase MPointer<T>

- **Implementación:** Se creó una clase genérica MPointer<T> que encapsula toda la lógica de comunicación con el servidor para crear, modificar, leer y eliminar bloques de memoria.
- **Alternativas consideradas:** Usar punteros inteligentes (std::shared_ptr, std::weak_ptr) como base, pero fueron descartados porque no permiten control distribuido sobre la memoria.
- **Limitaciones:** Solo soporta tipos primitivos (int, float, double, string). No se permite almacenamiento directo de estructuras complejas o punteros a memoria local.
- **Problemas encontrados:** Conversión de tipos a string para su transporte fue problemática para tipos no soportados. Se manejó mediante std::to_string() y especializaciones condicionales.

Método MPointer<T>::New()

- **Implementación:** Se definió static MPointer<T> New() como un método de fábrica que retorna un nuevo objeto MPointer<T>, equivalente a un constructor simplificado.
- **Ventajas:** Brinda una forma estándar y clara de crear instancias.
- **Limitaciones:** No ofrece ventajas funcionales sobre el constructor, pero sí mejora la semántica en el uso.

Manejo de referencias

- **Implementación:** Se maneja un conteo de referencias a través de llamadas a IncreaseRefCount y DecreaseRefCount en el servidor. Si el conteo llega a cero, el bloque se elimina.
- **Alternativas consideradas:** Control de referencias del lado del cliente (más complejo).
- **Problemas encontrados:** Olvido de decrementos en algunos casos. Se resolvió añadiendo la lógica dentro del destructor de MPointer.

Recolección de basura

- **Implementación:** Se utilizó un hilo (std::thread) en el servidor que corre cada 10 segundos y elimina bloques con ref_count == 0.
- **Limitaciones:** Es pasivo y periódico. Podría optimizarse con eventos.
- **Observación:** Se integra automáticamente sin intervención del cliente.

Desfragmentación de memoria

- **Implementación:** Luego de eliminar bloques sin referencias, se recorre la memoria y se reacomoda los bloques para compactarla, actualizando offsets.
- **Problemas encontrados:** Posibles colisiones al mover datos. Se resolvió usando memmove.

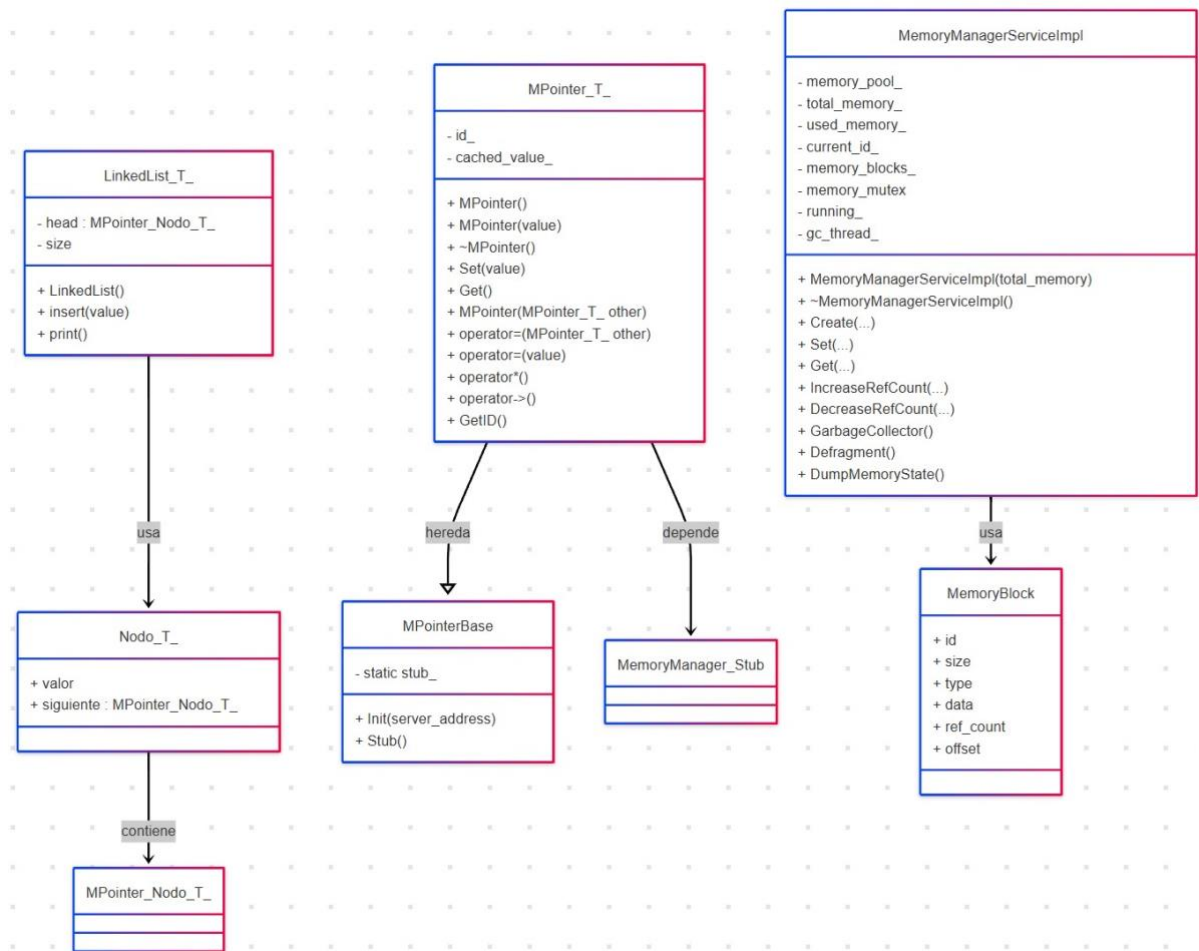
Volcado de memoria (DumpMemoryState)

- **Implementación:** Luego de cada acción que modifica la memoria (Create, Set, Increase/DecreaseRefCount, GC, etc.), se genera un archivo .txt en un directorio dumps/ con el estado actual de todos los bloques y sus atributos.
- **Formato:** El nombre del archivo incluye el timestamp en milisegundos.
- **Limitaciones:** Puede generar muchos archivos si hay muchas operaciones.
- **Valor agregado:** Permite depuración, análisis histórico, demostración visual y validación de memoria.

Sobrecarga de operadores

- **Implementación:**
 - operator*() para obtener el valor directamente (*mp = 5;),
 - operator->() para tipos estructurados,
 - operator= tanto para copiar un MPointer como para asignar un valor.
- **Problemas encontrados:** Conflictos con instanciación de plantillas y resolución de tipos.
- **Resultado:** Interfaz mucho más intuitiva y natural para el usuario final.

Diagrama UML



Link del repositorio de Github

<https://github.com/Alex21FW/MPointers2.0-2ND>